

Tugas Besar 2 IF3270 Pembelajaran Mesin CNN, RNN dan LSTM



Ditulis Oleh:

13523005	Muhammad Alfansya
13523035	M. Rayhan Farrukh
13523041	Hanif Kalyana Aditya

**Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
2026**

Daftar Isi

Daftar Isi.....	1
1. Deskripsi Persoalan.....	2
2. Pembahasan.....	3
2.1. Penjelasan Implementasi.....	3
2.1.1. Base.....	3
2.1.2. CNN.....	5
2.1.3. LSTM.....	9
2.1.4. RNN.....	12
2.1.5. Utils.....	14
2.2. Penjelasan Forward Propagation.....	16
2.2.1. CNN.....	16
2.2.2. RNN.....	16
2.2.3. LSTM.....	16
2.3. Hasil Pengujian.....	17
2.3.1. CNN.....	17
2.3.2. Simple RNN dan LSTM untuk Image Captioning.....	23
3. Kesimpulan dan Saran.....	33
3.1. Kesimpulan.....	33
3.2. Saran.....	33
4. Pembagian Tugas.....	34
5. Referensi.....	35
6. Lampiran.....	36

1. Deskripsi Persoalan

Persoalan utama dalam tugas ini adalah mengimplementasikan arsitektur Convolutional Neural Network (CNN) dan Recurrent Neural Network (RNN) beserta variannya, Long Short-Term Memory (LSTM), dari awal (from scratch), yang mencakup pembangunan struktur forward propagation secara modular untuk komponen spasial seperti layer konvolusi dan pooling, hingga pemrosesan sekuensial seperti layer embedding dan recurrent cell. Persoalan ini mengeksplorasi pengaruh hyperparameter pada arsitektur CNN, seperti jumlah layer konvolusi, ukuran dan banyak filter, jenis pooling layer, serta menganalisis efisiensi penggunaan parameter yang dibagikan (shared parameter) dibandingkan dengan parameter independen (non-shared). Pada arsitektur RNN dan LSTM, persoalan ini mengeksplorasi pengaruh jumlah layer recurrent dan ukuran hidden state, variasi panjang maksimum caption, serta pengujian perbandingan hasil forward propagation dari implementasi from scratch dengan prediksi yang menggunakan library Keras. Seluruh implementasi tersebut kemudian diaplikasikan pada dua domain berbeda: klasifikasi citra pada dataset Intel Image Classification dan sistem image captioning terintegrasi pada dataset Flickr8k menggunakan arsitektur encoder-decoder berbasis metode pre-inject untuk mengevaluasi kemampuan model dalam mempertahankan memori dan membangkitkan kalimat bermakna yang diukur menggunakan metrik BLEU-4 dan METEOR score.

2. Pembahasan

2.1. Penjelasan Implementasi

2.1.1. Base

Modul dense

Kelas Dense mengimplementasikan fully-connected layer generik yang digunakan di dua konteks berbeda dalam pipeline. Sebagai projection layer, ia memproyeksikan fitur CNN ke ruang embedding sebelum disuntikkan ke LSTM. Sebagai output layer, ia menghasilkan distribusi probabilitas atas seluruh vocabulary dengan aktivasi Softmax.

Berikut adalah rincian kelas-kelas yang ada pada modul ini:

Dense		
Atribut		
weights	shape (input_dim, output_dim)	Matriks bobot W yang di-load dari Keras, digunakan dalam operasi $x @ W$
bias	shape (output_dim,)	Vektor bias b yang ditambahkan setelah operasi linear
activation	Activation	Fungsi aktivasi yang diterapkan setelah operasi linear. Default Linear (identitas) untuk projection layer, Softmax untuk output layer
Metode		
forward(x)		menghitung $\text{activation}(x @ W + b)$ yang dipakai di dua tempat: sebagai projection layer (aktivasi Linear) dan output layer (aktivasi Softmax)

Modul Embedding

Kelas Embedding mengimplementasikan layer lookup table yang memetakan token integer ke representasi vektor padat. Berbeda dengan layer linear, tidak ada operasi aritmatika di sini karena forward pass cukup mengindeks baris ke-id dari matriks bobot, sehingga komputasinya sangat efisien.

Berikut adalah rincian kelas-kelas yang ada pada modul ini:

Embedding	
Atribut	

embeddings	shape (vocab_size, embed_dim)	matriks bobot yang di-load dari Keras. Dua method
vocab_size	int	Jumlah total kata unik dalam kosakata, termasuk token spesial, yang merepresentasikan dimensi baris dari matriks <i>embedding</i> .
embed_dim	int	Ukuran dimensi dari vektor representasi padat untuk setiap kata, yang merepresentasikan dimensi kolom dari matriks <i>embedding</i> .
Metode		
<code>forward(x)</code>		Mengeksekusi langkah forward pass dengan melakukan pencarian indeks token langsung pada matriks bobot embeddings

Modul Flatten

Modul flatten berfungsi untuk mengubah bentuk tensor input multidimensi menjadi tensor dua dimensi agar dapat diproses oleh layer fully connected (dense).

Berikut adalah rincian kelas-kelas yang ada pada modul ini:

Flatten	
Metode	
<code>forward</code>	Mengubah dimensi tensor input menjadi vektor linier (flat) dengan mempertahankan ukuran batch

Modul Activations

Modul yang menyimpan fungsi-fungsi aktivasi yang akan digunakan pada model-model yang diimplementasikan. Dibuat sebagai abstract base class yang diinherit oleh masing-masing fungsi aktivasi

Berikut adalah rincian kelas-kelas yang ada pada modul ini:

Activation
Metode

forward	Mengaplikasikan fungsi aktivasi tertentu terhadap data
Fungsi aktivasi yang ada	
Linear	
ReLU	
Sigmoid	
Softmax	

2.1.2. CNN

Modul Layers

Layers pada CNN scratch diimplementasikan sebagai abstract base class, yang akan diinherit oleh kelas Conv2D, dan LocallyConnected2D. Penggunaan abstract base class disini agar bisa berbagi method antara kedua jenis layer.

Berikut adalah rincian kelas-kelas yang ada pada modul ini:

CNNLayer	
Atribut	
strides	Menyimpan nilai stride
padding	Menyimpan mode padding, yaitu 'valid' atau 'same'
activation	Menyimpan fungsi aktivasi yang digunakan pada layer
Metode	
_get_fmap_dims	Menghitung dimensi output pada layer berdasarkan dimensi input, nilai stride, padding, dan juga kernel
_pad	Menambah nilai nol ke tepi berdasarkan suatu konfigurasi padding
forward	Abstract method untuk forward propagation
Conv2D	
Atribut	
kernel	Menyimpan filter yang berisikan bobot yang terbagi

	untuk seluruh posisi spasial
bias	Menyimpan nilai bias yang akan ditambah ke output channel
Metode	
forward	Forward propagation, mengambil patch gambar dan melakukan dot product terhadap kernel
LocallyConnected2D	
Atribut	
kernel	Menyimpan filter yang berisikan bobot yang independen untuk setiap posisi output.
bias	Menyimpan bias yang berbeda untuk setiap posisi output
Metode	
forward	Forward propagation, mengambil patch gambar dan melakukan dot product terhadap kernel sesuai posisi
_get_kernel_size	Menghitung ukuran kernel berdasarkan shape bobot

Modul Model

Modul utama CNN, berfungsi menyusun layer-layer menjadi sebuah arsitektur, serta melakukan pemuatan bobot. Selain itu juga sebagai kelas utama yang pengguna gunakan untuk melakukan forward propagation dan prediction.

Berikut adalah rincian kelas-kelas yang ada pada modul ini:

Activation	
Atribut	
spec	List dictionary yang mendefinisikan sebuah arsitektur, dengan menyimpan informasi seperti nama sebuah layer dan ukurannya.
layers	List yang menyimpan layer-layer yang sudah diinstansiasikan
Metode	

<code>forward</code>	Melakukan forward pass sekuensial melalui setiap layer yang ada di model
<code>prediction</code>	Melakukan prediksi terhadap data, dengan memanggil forward propagation terhadap batch gambar
<code>count_params</code>	Menghitung parameter yang ada pada model sesuai parameter yang ada di tiap layer
<code>init_activation</code>	Menginstansiasikan kelas aktivasi yang digunakan sesuai pilihan pengguna
<code>_extract_weights_from_spec</code>	Memuat bobot langsung dari spec, bukan dari model keras. Digunakan untuk LocallyConnected2D

Modul Pooling

Modul yang berisi kelas-kelas layer untuk melakukan reduksi dimensi spasial pada feature map, yaitu operasi Pooling. Diimplementasikan sebagai abstract base class, yang diinherit oleh semua Pooling lainnya, yaitu max dan average pooling, untuk lokal dan global.

Berikut adalah rincian kelas-kelas yang ada pada modul ini:

Pooling	
Atribut	
-	
Metode	
<code>forward</code>	Menjalankan forward pass pooling (method abstrak)
LocalPooling	
Atribut	
<code>pool_size</code>	Tuple untuk menyimpan ukuran filter pooling yang digunakan
<code>strides</code>	Menyimpan nilai stride filter pooling
Metode	
<code>forward</code>	Menjalankan forward pass pooling (method abstrak)

GlobalPooling	
Atribut	
-	
Metode	
-	
MaxPooling2D	
Atribut	
-	
Metode	
forward	Menjalankan pooling dengan mengambil nilai maksimum setiap patch lokal
AveragePooling2D	
Atribut	
-	
Metode	
forward	Menjalankan pooling dengan mengambil nilai rata-rata setiap patch lokal
GlobalMaxPooling2D	
Atribut	
-	
Metode	
forward	Menjalankan pooling dengan mengambil nilai maksimum dari keseluruhan dimensi spasial pada channel
GlobalAveragePooling2D	
Atribut	
-	
Metode	

<code>forward</code>	Menjalankan pooling dengan mengambil nilai rata-rata dari keseluruhan dimensi spasial pada channel
----------------------	--

2.1.3. LSTM

Modul Embedding

Kelas Embedding mengimplementasikan layer lookup table yang memetakan token integer ke representasi vektor padat. Berbeda dengan layer linear, tidak ada operasi aritmatika di sini karena forward pass cukup mengindeks baris ke-id dari matriks bobot, sehingga komputasinya sangat efisien.

Embedding		
Atribut		
<code>W</code>	<code>shape (vocab_size, embed_dim)</code>	matriks bobot yang di-load dari Keras. Dua method
Metode		
<code>load_weights(weights)</code>		Digunakan untuk mengekstrak tensor embeddings dari dict hasil <code>weight_loaders</code>
<code>forward(token_ids)</code>		operasinya cukup return <code>self.W[token_ids]</code> , yaitu index lookup sederhana

Modul lstm_cell

Modul normalizers menyimpan implementasi abstract base class metode-metode normalisasi beserta kelas-kelas konkritnya. Adapun kelas normalisasi konkrit yang kami implementasikan hanyalah RMSNorm untuk Root Mean Square Normalization.

LSTMCell	
Atribut	
<code>W_xi, W_hi, b_i</code>	Input gate
<code>W_xf, W_hf, b_f</code>	Forget gate
<code>W_xc, W_hc, b_c</code>	Candidate

<code>W_xo, W_ho, b_o</code>	Output gate
Metode	
<code>load_weights(weights)</code>	Keras menyimpan semua gate dalam satu matrix besar (kernel shape (input_dim, 4H), recurrent_kernel shape (H, 4H), bias shape (4H,)).
<code>forward(x, H, C)</code>	menghitung persamaan LSTM dan return H_new, C_new masing-masing shape (batch_size, H)

Modul Istm_decoder

Kelas LSTMDecoder merakit seluruh komponen LSTM menjadi pipeline image captioning end-to-end menggunakan metode pre-inject. Fitur CNN diproyeksikan ke ruang embedding dan digunakan sebagai input pertama sebelum token <start>, sehingga informasi visual tersedia sejak awal proses decoding.

LSTMDecoder		
Atribut		
embedding	Embedding	Layer embedding untuk mengkonversi token caption menjadi dense vector
dense_proj	Dense	Layer proyeksi yang mengkonversi fitur CNN (dim 2048) ke ruang embedding (dim 256), digunakan sebagai input x_{-1}
embedding, dense_proj	LSTMCell	LSTM cell tunggal yang memproses satu timestep sekaligus, menerima input dan state sebelumnya
dense_out	Dense	Layer output dengan aktivasi Softmax yang mengkonversi hidden state ke distribusi probabilitas atas vocabulary
Metode		
<code>load_weights(weights_dict)</code>	Mendistribusikan bobot ke masing-masing komponen berdasarkan field target di spec	
<code>forward(cnn_features, token_sequences)</code>	full batch forward pass mode teacher forcing, return shape (batch, seq_len+1, vocab_size)	
<code>generate_caption(cnn_feature, word2idx, idx2word, max_len)</code>	decoding untuk satu gambar: inject CNN → feed <start> → argmax berulang hingga <end> atau max_len	

max_len)	
----------	--

Modul lstm_scratch

Kelas LSTMScratch mengimplementasikan pipeline image captioning end-to-end dari raw image hingga caption menggunakan seluruh komponen from scratch. Kelas ini menggabungkan CNN encoder pretrained (InceptionV3) untuk ekstraksi fitur dengan LSTMDecoder from scratch yang bobotnya di-load dari hasil training Keras, sehingga seluruh proses inference berjalan tanpa ketergantungan pada framework deep learning.

LSTMScratch		
Atribut		
Nama	Tipe	Keterangan
meta	dict	Metadata vocabulary dari metadata.json, berisi vocab_size, max_seq_len, start_idx, end_idx, dll
encoder	keras.Model	InceptionV3 pretrained (frozen, tanpa top layer) untuk ekstraksi fitur gambar
preprocess_fn	callable	Fungsi preprocessing gambar dari [0,1] ke [-1,1] sesuai format InceptionV3
target_size	tuple	Ukuran input gambar untuk encoder, (299, 299) untuk InceptionV3
dense_proj	Dense	Layer proyeksi CNN feature ke embed_dim, bobot di-load dari Keras
embedding	Embedding	Layer embedding untuk token caption, bobot di-load dari Keras
lstm_cells	list[LSTMCell]	List LSTM cell per layer — mendukung multi-layer (1, 2, atau 3 layer)
dense_out	Dense	Layer output dengan Softmax, bobot di-load dari Keras
Metode		
__init__(keras_model_path, metadata_path, encoder_name)		Inisialisasi pipeline dengan load semua komponen. Membangun CNN encoder, load Keras model, lalu ekstrak bobot tiap layer ke komponen scratch yang sesuai. LSTM cells

	di-load secara dinamis dengan iterasi seluruh layer Keras dan memfilter yang bertipe keras.layers.LSTM, sehingga otomatis mendukung variasi 1, 2, maupun 3 layer.
<code>generate_caption(image_path, idx_to_word, max_len)</code>	Menghasilkan caption untuk satu gambar menggunakan greedy decoding end-to-end, mulai dari raw image file hingga string caption.

2.1.4. RNN

Modul RNN_cell

Kelas RNN_Cell mengimplementasikan lapisan SimpleRNN dari scratch untuk memproses data sekuensial berbasis waktu. Modul ini bertanggung jawab untuk mempertahankan memori kontekstual sepanjang rangkaian sekuens dengan cara memperbarui nilai hidden state secara bertahap pada setiap langkah waktu. Modul ini memanfaatkan operasi matriks tervektor pada NumPy untuk mendukung penuh fungsionalitas batch inference secara efisien.

RNN_Cell	
Atribut	
<code>W_xh</code>	Matriks bobot kernel berukuran (input_dim, units) yang memetakan vektor input x_t pada timestep aktif ke dalam hidden state
<code>W_hh</code>	Matriks bobot recurrent berukuran (units, units) yang memetakan memori hidden state sebelumnya $h_{(t-1)}$ ke timestep aktif.
<code>b_h</code>	Vektor bias berukuran (units,)
<code>units</code>	Jumlah dimensi atau banyak hidden units
Metode	
<code>forward_step(x_t, h_prev)</code>	Menghitung hidden state baru dengan fungsi aktivasi tanh.
<code>forward(x, h_prev, return_sequences)</code>	Mengeksekusi proses forward pass untuk seluruh rangkaian sekuens input. Metode ini menghasilkan keluaran berupa seluruh urutan memori jika <code>return_sequences=True</code> , atau hanya menghasilkan memori pada timestep terakhir jika bernilai False.

RNNKeras

Kelas RNNKeras mengimplementasikan pipeline image captioning berbasis Keras untuk digunakan sebagai model acuan. Modul ini mengintegrasikan seluruh tahapan pemrosesan tingkat tinggi secara sekuensial, mulai dari pemuatan dan preprocess citra mentah, ekstraksi vektor fitur spasial menggunakan *pretrained* CNN *encoder* pada Keras *decoder* yang telah dilatih. Modul ini berfungsi untuk mengevaluasi kualitas *captioning* resmi serta menjadi pembanding performa dan waktu eksekusi terhadap implementasi arsitektur *from scratch*.

RNNKeras		
Atribut		
meta	dict	Dictionary yang menyimpan metadata konfigurasi kosakata dan sekuens (max_seq_len, start_idx, end_idx, dan pad_idx)
encoder	keras.Model	Model CNN <i>pretrained</i> (InceptionV3)
preprocess_fn	function	Fungsi preprocessing gambar dari [0,1] ke [-1,1] sesuai format InceptionV3
target_size	tuple	Ukuran input gambar untuk encoder, (299, 299) untuk InceptionV3
decoder	keras.Model	Layer output dengan aktivasi Softmax yang mengkonversi hidden state ke distribusi probabilitas atas vocabulary
Metode		
generate_caption(image_path, idx_to_word, max_len)		Memuat gambar, melakukan normalisasi piksel, mengekstrak fitur gambar 1D, serta menjalankan <i>decoding</i> berbasis <i>teacher forcing</i> pada input tensor untuk menghasilkan sekuens <i>caption</i> .

RNNScratch

Kelas RNNScratch mengimplementasikan pipeline image captioning secara mandiri menggunakan komponen yang dibuat from scratch. Modul ini menjembatani bagian encoder spasial (Keras pretrained CNN) dengan rangkaian decoder

sekuensial berbasis NumPy murni melalui weight loading hasil pelatihan eksternal. Secara arsitektural, modul ini mengadopsi metode pre-inject (Show and Tell).

RNNScratch		
Atribut		
Nama	Tipe	Keterangan
meta	dict	Dictionary yang menyimpan metadata konfigurasi kosakata dan sekuens (max_seq_len, start_idx, end_idx, dan pad_idx)
encoder	keras.Model	Model CNN <i>pretrained</i> (InceptionV3)
preprocess_fn	function	Fungsi preprocessing gambar dari [0,1] ke [-1,1] sesuai format InceptionV3
target_size	tuple	Ukuran input gambar untuk encoder, (299, 299) untuk InceptionV3
dense_proj	Dense	Layer proyeksi CNN feature ke embed_dim, bobot di-load dari Keras
embedding	Embedding	Layer embedding untuk token caption, bobot di-load dari Keras
rnn_cells	list[RNNCell]	List RNN cell per layer, mendukung multi-layer (1, 2, atau 3 layer)
dense_out	Dense	Layer output dengan Softmax, bobot di-load dari Keras
Metode		
<code>__init__(keras_model_path, metadata_path, encoder_name)</code>		Memuat berkas konfigurasi JSON, menginisialisasi CNN encoder, mengekstrak struktur bobot dari model .keras secara otomatis, dan memindahkannya ke dalam instans objek layer from scratch (Dense, Embedding, dan sel RNN).
<code>generate_caption(image_path, idx_to_word, max_len)</code>		Mengeksekusi pipa inferensi from scratch: membaca citra, mengekstrak fitur visual, menyuntikkan fitur sebagai pemicu memori awal pada seluruh layer RNN, lalu menjalankan looping greedy decoding untuk membuat caption

2.1.5. Utils

Modul Image Utils

Modul yang berisikan fungsi-fungsi untuk memuat gambar, yang terdiri dari dua fungsi yaitu sebagai berikut.

- `load_image` membuka sebuah gambar, memastikan warna RGB, dan melakukan resizing sesuai resolusi target. Hasil akhir dikonversi menjadi array NumPy dan dinormalisasi ke rentang nilai 0-1.
- `load_batch` memproses sekumpulan gambar secara sekuensial menggunakan `load_image`. Seluruh array gambar kemudian digabungkan pada dimensi baru untuk menghasilkan satu tensor berdimensi empat dengan format (N, H, W, C).

Modul Feature Extractor

Modul untuk melakukan ekstraksi representasi fitur dari kumpulan data citra menggunakan model enkoder tertentu. Fungsi pada modul ini memeriksa keberadaan cache, jika ditemukan, fungsi langsung memuat dan mengembalikan fitur yang sudah ada. Jika belum ada, fungsi memecah daftar citra ke dalam beberapa batch, memuatnya ke memori, dan mengekstrak fiturnya melalui prediksi model. Seluruh hasil blok fitur digabungkan menjadi satu array utuh, disimpan ke dalam jalur *cache* baru, lalu dikembalikan sebagai output.

Modul Weight Loader

Modul ini berisikan fungsi `load_weights`, yang berfungsi mengekstrak dan memetakan bobot dari model Keras ke struktur dictionary. Fungsi bekerja dengan mengiterasi setiap entri konfigurasi lapisan (*spec*). Pada setiap iterasi, fungsi mengambil objek lapisan dari model Keras asli melalui pencarian nama, dan mencocokkan tipe lapisan dengan daftar *weight keys* internal yang valid, kemudian mengekstrak nilai tensor parameter seperti *kernel* dan *bias*. Nilai tersebut dikonversi menjadi array NumPy dan disusun ke dalam sebuah *nested dictionary* terstruktur yang memuat tipe, bobot, serta metadata terkait.

Modul `extract_flickr8k_features`

Modul ini merupakan script khusus untuk mengekstrak CNN feature vectors dari seluruh dataset Flickr8k secara per-split (train/val/test). Script ini menggabungkan dua implementasi sebelumnya dan menggunakan `extract_features()` dari `feature_extractor` sebagai fungsi inti, dengan tambahan dukungan dua encoder dan preprocessing yang benar untuk masing-masing model.

Metode	
<code>build_encoder(model_name) -> (encoder, preprocess_fn, target_size)</code>	Membangun CNN encoder pretrained dari Keras tanpa top layer dan membekukan semua bobotnya. Mengembalikan tiga hal sekaligus: model encoder, fungsi preprocessing yang mengkonversi batch [0,1] ke format yang sesuai model, dan ukuran input gambar yang diperlukan.
<code>load_split_paths(split_file, images_dir) -> (paths, image_ids)</code>	Membaca file split Flickr8k (.txt) dan mengembalikan list path lengkap gambar beserta image id-nya. Juga melakukan validasi bahwa semua file gambar benar-benar ada di disk.
<code>main(images_dir, split_dir, output_dir, model_name, batch_size)</code>	Fungsi utama yang mengorkestrasi ekstraksi fitur untuk ketiga split. Untuk setiap split, memanggil <code>extract_features()</code> dengan <code>preprocess_fn</code> yang sesuai dan menyimpan hasilnya ke <code>outputs/features/</code> dengan prefix nama model agar tidak saling menimpa antar encoder.

Modul generate_splits

Script bantuan yang dijalankan sekali untuk membuat file split Flickr8k. Dataset Flickr8k dari Kaggle tidak menyertakan file split resmi, sehingga script ini membuat tiga file split dengan pembagian 6000/1000/1000 untuk train/val/test secara reproducible menggunakan random seed tetap. Selain itu, script ini hanya perlu dijalankan sekali. Hasil split yang sama digunakan bersama oleh LSTM dan RNN agar perbandingan keduanya fair.

Metode	
<code>main(images_dir, output_dir)</code>	Mengumpulkan semua file .jpg di folder gambar, melakukan shuffle dengan seed tetap (seed=42) agar reproducible, lalu membagi dan menyimpan ke tiga file teks.

2.2. Penjelasan Forward Propagation

2.2.1. CNN

Forward propagation pada Conv2D diawali dengan penyetaraan dimensi spasial gambar input terhadap ukuran filter yang digunakan. Jika konfigurasi untuk mempertahankan ukuran gambar diaktifkan, tepi gambar akan diberikan padding simetris, sebaliknya jika dinonaktifkan, gambar akan diproses apa adanya. Selanjutnya, sebuah sliding window spasial lokal akan bergerak menyusuri permukaan gambar dari kiri ke kanan dan dari atas ke bawah dengan jarak lompatan (stride) tertentu sesuai konfigurasi. Pada setiap posisi, patch lokal gambar dikalikan elemen demi elemen dengan struktur filter bobot yang sama untuk seluruh area gambar, lalu diakumulasikan di seluruh input channel. Hasil transformasi spasial bobot yang terbagi (shared weight) ini kemudian ditambahkan dengan nilai bias sebelum dilewatkan ke fungsi aktivasi feature map baru untuk tahap berikutnya.

Sementara itu, proses forward propagation pada LocallyConnected2D memiliki kemiripan dalam hal pemindaian spasial, namun tanpa kesamaan bobot di setiap areanya. Proses dimulai dengan memotong gambar menjadi serangkaian sliding window menggunakan aturan stride tertentu langsung pada gambar tanpa adanya padding. Perbedaan terjadi pada tahap komputasi, di mana lapisan ini telah menyediakan serangkaian filter bobot yang unik untuk setiap koordinat posisi spasial pada hasil pemetaan. Setiap potongan wilayah gambar pada lokasi tertentu akan dikalikan secara eksklusif dengan filter khusus yang hanya dialokasikan untuk posisi tersebut, sehingga karakteristik fitur yang dihasilkan sangat bergantung pada posisinya di dalam gambar. Kombinasi linear unik per posisi ini kemudian ditambahkan dengan nilai bias yang juga bersifat spesifik untuk tiap koordinat, lalu diakhiri fungsi aktivasi untuk menentukan nilai final dari feature map tersebut.

2.2.2. RNN

Proses forward propagation pada kelas RNN diawali dengan tahapan ekstraksi fitur visual dan inisialisasi memori awal. Gambar mentah diproses melalui Keras pretrained CNN encoder untuk menghasilkan vektor fitur spasial, yang kemudian diproyeksikan melewati lapisan linear dense_proj menjadi vektor representasi berukuran dimensi embedding. Sebelum rangkaian kata diproses, memori internal atau hidden state untuk setiap tingkatan lapisan SimpleRNN diinisialisasi dengan nilai nol (h_t_list). Pada timestep pemicu ($t = -1$), vektor proyeksi gambar tersebut diumpukan ke dalam tumpukan rnn_cells menggunakan metode forward_step secara

berurutan, di mana hasil pembaruan hidden state dari lapisan terdahulu langsung menjadi masukan bagi lapisan berikutnya, sehingga berhasil menyuntikkan konteks visual ke dalam memori internal seluruh sel RNN.

Setelah memori awal terbentuk, model melanjutkan forward pass ke dalam tahapan perulangan pembangkitan teks dari timestep $t = 0$ hingga batas panjang maksimum sekuens. Proses dimulai dengan memasukkan token penanda awal (*start_idx*) ke dalam lapisan embedding untuk diekstraksi menjadi dense vector. Vektor kata ini kemudian dialirkan melewati setiap lapisan di dalam *rnn_cells* secara sekuensial, di mana metode *forward_step* memperbarui hidden state aktif berdasarkan kombinasi masukan saat ini dan memori timestep sebelumnya. Hasil akhir dari lapisan RNN teratas selanjutnya diumpankan ke dalam lapisan linear output *dense_out* beraktivasi Softmax untuk menghitung distribusi probabilitas seluruh kata dalam kosakata. Kata dengan peluang tertinggi diekstrak, disimpan ke dalam daftar hasil prediksi, dan langsung dijadikan sebagai token masukan baru untuk mengulang seluruh proses forward propagation pada timestep berikutnya hingga token penutup (*end_idx*) ditemui.

2.2.3. LSTM

Forward propagation LSTM dalam konteks image captioning mengikuti arsitektur *pre-inject* dari [paper \(Tanti et.al., 2017\)](#) yang seolah-olah menganggap gambar sebagai kata pertama dalam *prefix*. Maka, fitur CNN di-*inject* sebagai input pertama sebelum token *<start>*, lalu LSTM memproses sequence secara *autoregressive* untuk menghasilkan distribusi probabilitas atas vocabulary di setiap timestep. Sementara itu, dalam proses *forward propagation*-nya, LSTM terbagi dari beberapa tahap sebagai berikut :

- a. Tahap 1 : Proyeksi Fitur CNN (Dense Projection)
Fitur CNN yang dihasilkan dari proses ekstraksi menggunakan *pre-trained* dari Keras yaitu InceptionV3 berukuran (1, 2048) diproyeksikan ke ruang embedding berukuran (1, 256) melalui layer Dense. Hasil proyeksi ini menjadi input pertama ke LSTM pada timestep $t = -1$, sebelum token *<start>*.
- b. Tahap 2 : Embedding Token
Setiap token caption dikonversi ke dense vector melalui operasi lookup di mana *W_emb* adalah matriks embedding berukuran (*vocab_size*, *embed_dim*).
- c. Tahap 3 : LSTM Cell (per timestep)
Ini adalah inti dari forward propagation. Pada setiap timestep t , LSTM menerima input x_t , hidden state sebelumnya $H_{\{t-1\}}$, dan cell state

sebelumnya C_{t-1} , lalu menghitung dan memperbarui gate/cell berikut ([slide kuliah IF3270 Pembelajaran Mesin 2025/2026](#)):

- Input gate (mengontrol informasi baru yang masuk)
- Forget gate (mengontrol informasi lama yang dilupakan)
- Output gate (mengontrol informasi yang dikeluarkan)
- Candidate cell state (informasi baru yang akan ditambahkan)
- Update cell state
- Update hidden state

d. Tahap 4 : Output Layer (Dense & Softmax)

Hidden state H_t diproyeksikan ke distribusi probabilitas atas seluruh vocabulary. Lalu, token berikutnya dipilih dengan argmax (greedy decoding)

2.3. Hasil Pengujian

2.3.1. CNN

2.3.1.1. Scratch vs Keras

Macro F1-Score Keras Shared (Saved) : 0.8219				
Macro F1-Score Scratch Shared : 0.6478				
Performance Difference : 0.174170				
Classification Report: Keras Shared				
	precision	recall	f1-score	support
buildings	0.75	0.89	0.81	437
forest	0.94	0.95	0.94	474
glacier	0.75	0.83	0.79	553
mountain	0.76	0.77	0.77	525
sea	0.86	0.70	0.77	510
street	0.89	0.81	0.85	501
accuracy			0.82	3000
macro avg	0.83	0.82	0.82	3000
weighted avg	0.83	0.82	0.82	3000
Classification Report: Scratch Shared				
	precision	recall	f1-score	support
buildings	0.63	0.85	0.72	437
forest	0.96	0.86	0.91	474
glacier	0.51	0.90	0.65	553
mountain	0.70	0.61	0.65	525
sea	0.88	0.10	0.18	510
street	0.80	0.76	0.78	501
accuracy			0.68	3000

macro avg	0.74	0.68	0.65	3000
weighted avg	0.74	0.68	0.64	3000

Model Keras menghasilkan performa yang lebih tinggi dibanding dengan Scratch, dengan selisih Macro F1-score sebesar 0.1741. Jika dilihat dari classification report, penurunan kualitas prediksi paling banyak terjadi pada kelas glacier (F1-score turun dari 0.79 ke 0.65) dan buildings (dari 0.81 ke 0.72), model scratch terlalu banyak menghasilkan false positive pada kedua kelas tersebut. Kelas forest cenderung stabil dengan hanya turun dari 0.94 menjadi 0.91, yang kemungkinan karena hutan lebih unik dibanding kategori gambar lainnya. Perbedaan hasil ini adalah karena implementasi forward propagation scratch, mengingat kedua model menggunakan arsitektur dan bobot yang sama.

2.3.1.2. 16 Variasi Hyperparameter

```

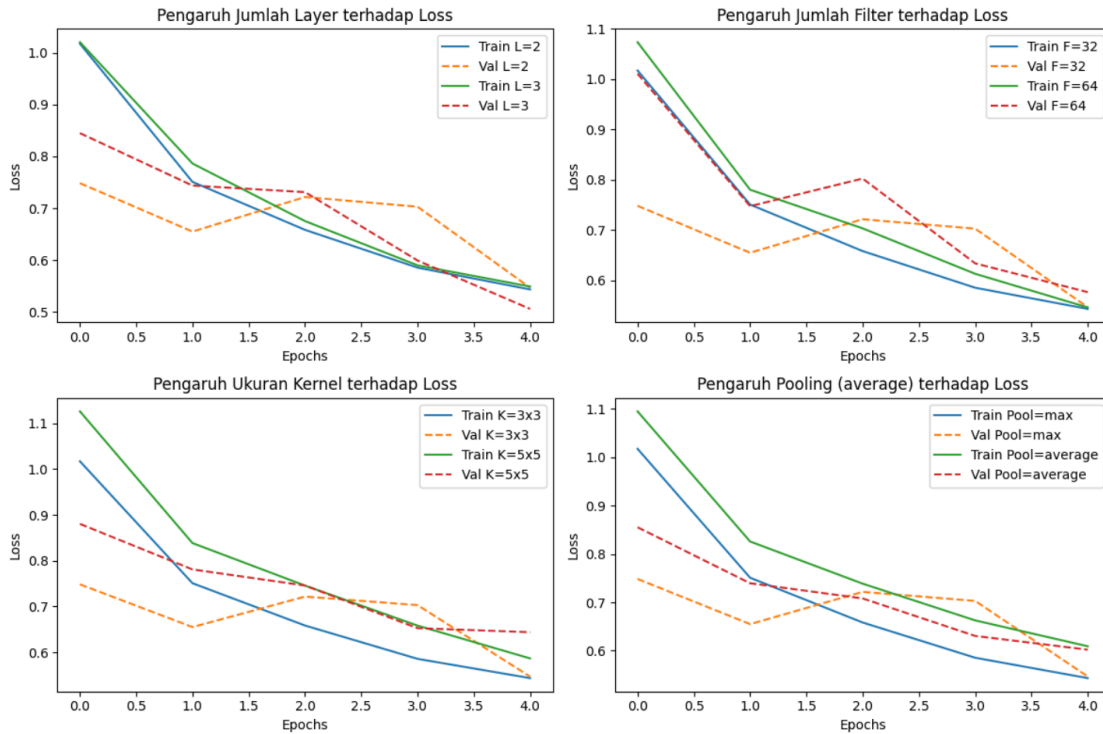
=== Average Macro F1: Layer Counts ===
n_layers
2      0.776066
3      0.792994

=== Average Macro F1: Filter Counts ===
n_filters
32      0.789748
64      0.779313

=== Average Macro F1: Kernel Size ===
kernel_size
3      0.801431
5      0.767630

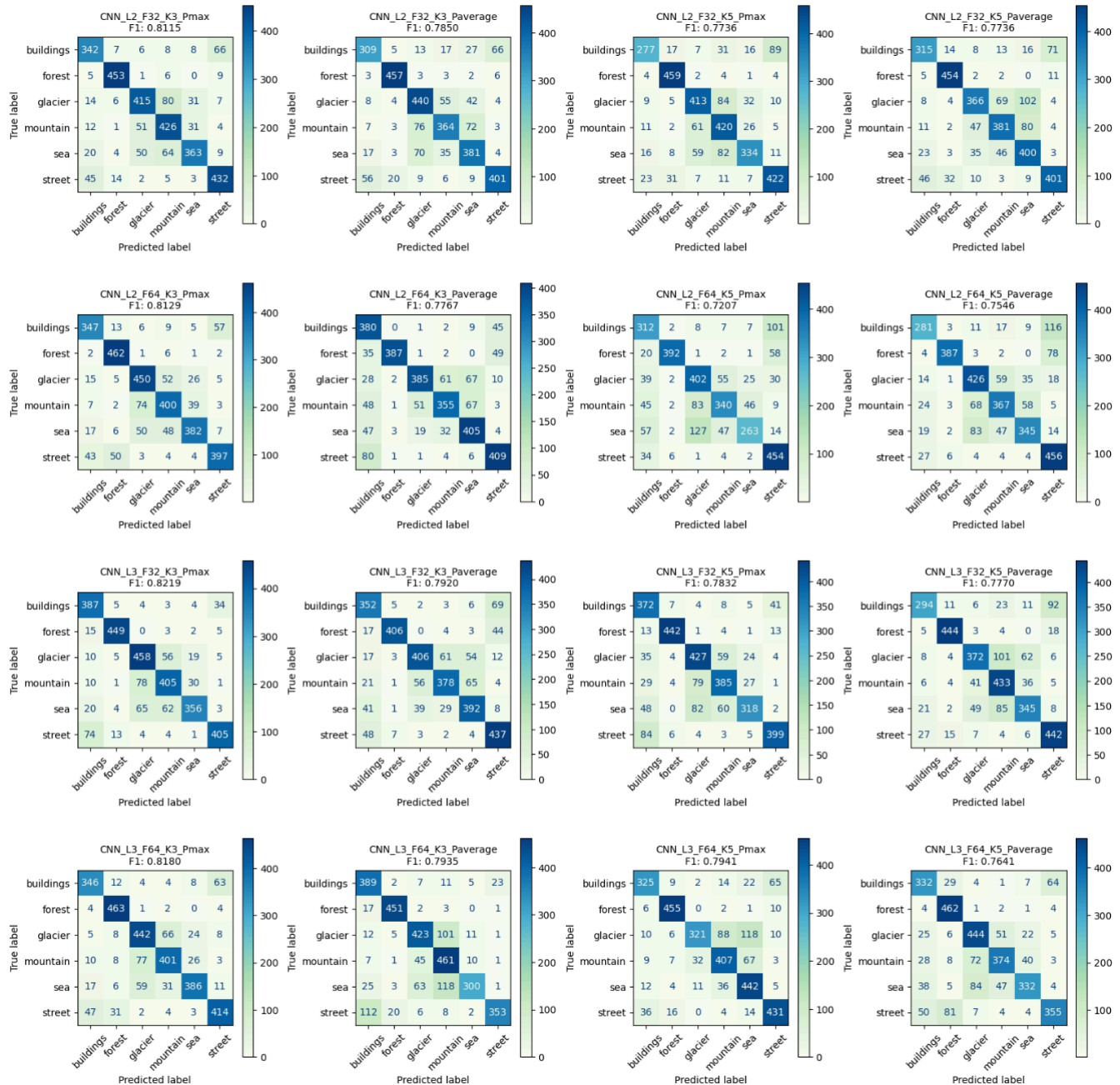
=== Average Macro F1: Pooling ===
pooling
average      0.777068
max          0.791993

```



Gambar Loss Curve Variasi Hyperparameter

Berdasarkan data rata-rata Macro F1-score dan loss curve, peningkatan jumlah layer konvolusi menjadi 3 layer (0.7930) dan penggunaan max pooling (0.7920) memberikan performa superior dibandingkan opsi lainnya. Loss curve mengonfirmasi bahwa kombinasi ini mempercepat konvergensi menuju error yang rendah dan stabil. Sebaliknya, penggunaan filter yang lebih banyak (64 filter) dan ukuran kernel yang lebih lebar (5×5) justru menurunkan performa rata-rata (menjadi 0.7793 dan 0.7676). Grafik loss memperlihatkan adanya fluktuasi dan perlambatan penurunan val loss pada filter 64 dan kernel 5×5, yang mengindikasikan bahwa kapasitas model yang terlalu besar memicu kejenuhan variansi fitur pada resolusi gambar dataset ini.

**Gambar** Confusion Matrix Variasi Hyperparameter

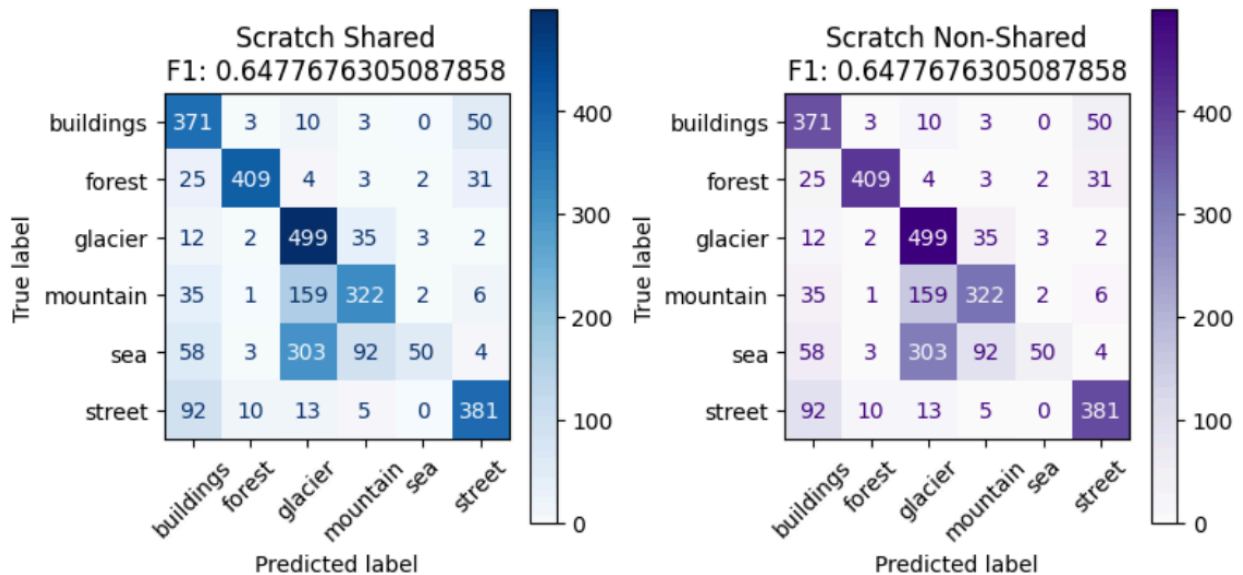
Jika ditinjau dari sebaran 16 confusion matrix, model dengan performa tertinggi adalah CNN_L3_F32_K3_Pmax (F1: 0.8219) dan yang terendah adalah CNN_L2_F64_K5_Pmax (F1: 0.7207). Di seluruh variasi arsitektur, terdapat pola misklasifikasi yang konsisten akibat kemiripan visual yang tinggi, terutama pada pertukaran kelas antara glacier dengan mountain, serta kelas buildings dengan street. Arsitektur terbaik dengan kernel 3×3

dan kedalaman 3 layer adalah yang paling efektif untuk meminimalkan error pada kelas-kelas tersebut dibandingkan arsitektur lainnya.

2.3.1.3. Shared vs Non-Shared

```

=== ANALISIS EVALUASI MODEL STRATCH ===
Macro F1-Score Shared           : 0.6478
Macro F1-Score Non-Shared       : 0.6478
Parameters for Shared           : 1,347,398
Parameters for Non-Shared       : 86,168,518
Parameters Ratio (Non-Shared/Shared) : 63.95x
  
```



Gambar Confusion Matrix Shared vs Non-Shared

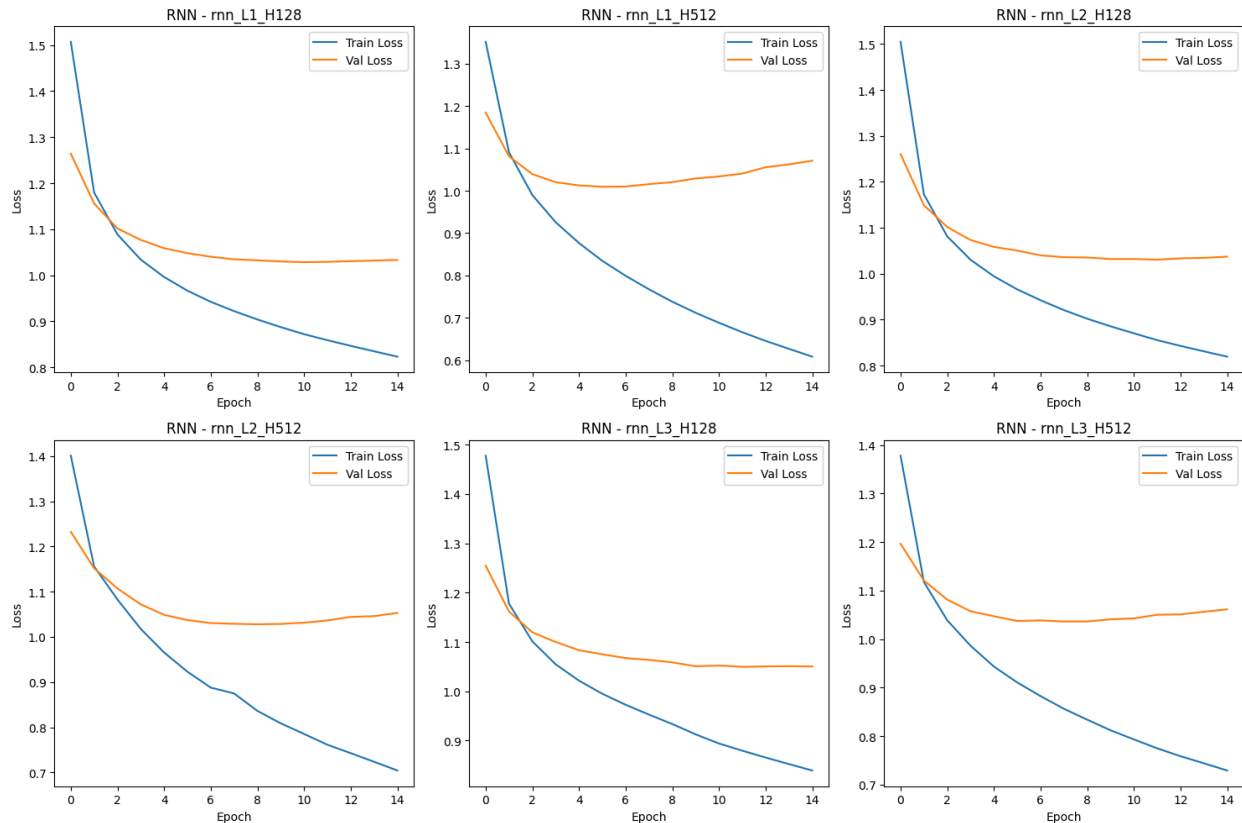
Model Scratch Shared dan Scratch Non-Shared menghasilkan nilai Macro F1-score yang sama (0.6478) serta distribusi klasifikasi pada confusion matrix yang identik. Kesamaan ini menunjukkan keberhasilannya metode tiling parameter. Ketika bobot filter dari arsitektur shared dipetakan secara statis ke koordinat spasial lokal model non-shared, karakteristik ekstraksi fitur lokal yang telah dilatih tetap terjaga seutuhnya, sehingga tidak mengubah keputusan prediksi akhir model pada data uji.

Perbedaan di antara keduanya terletak pada efisiensi parameter. Jumlah parameter pada model Non-Shared membengkak sebesar 63.95 kali lipat menjadi 86,168,518 parameter, dibandingkan model Shared yang hanya membutuhkan 1,347,398 parameter. Tentunya ini disebabkan oleh tiadanya weight sharing pada arsitektur non-shared, sehingga network mengalokasikan kernel dan bias yang berbeda untuk setiap titik koordinat spasial. Eksperimen ini menegaskan lagi bahwa penggunaan parameter

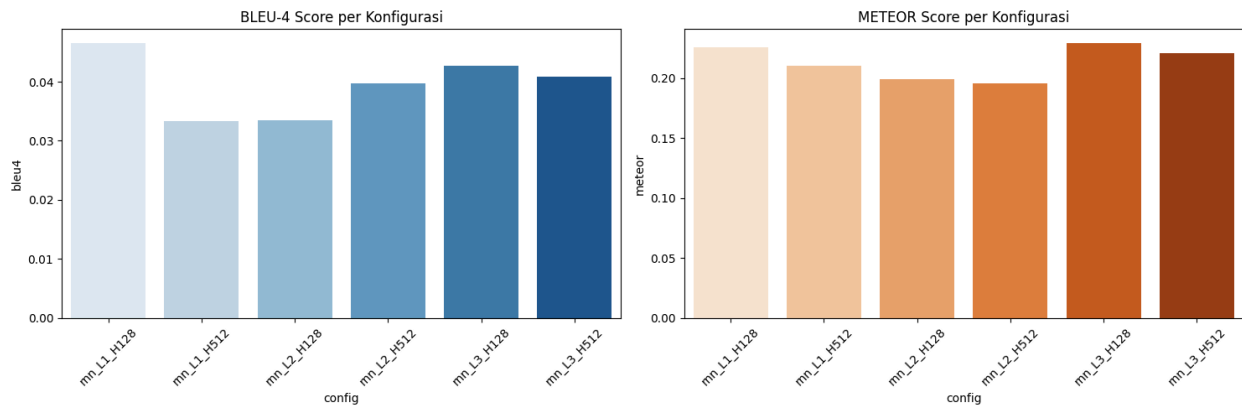
sharing pada layer konvolusi penting untuk memangkas redundansi parameter dan menghemat memori komputasi tanpa menurunkan kualitas generalisasi model.

2.3.2. Simple RNN dan LSTM untuk Image Captioning

2.3.2.1. Perbandingan jumlah layer: RNN



Gambar 2.1 Training vs Validation Loss RNN dengan 6 buah variasi



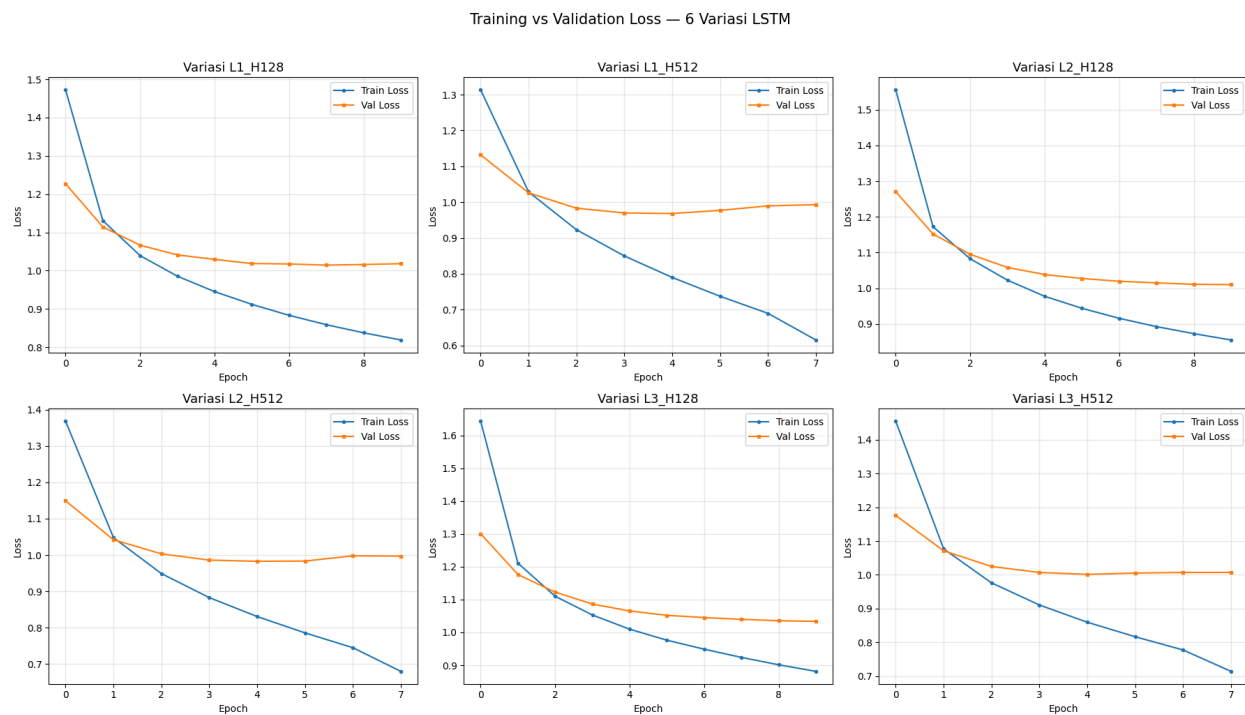
Gambar 2.2 Skor Bleu-4 dan METEOR per konfigurasi RNN

Berdasarkan gambar 2.1, Penambahan jumlah layer pada model dengan ukuran hidden state kecil (H128) mampu mempertahankan stabilitas kurva validation loss sehingga tetap landai hingga akhir epoch. Hasil pengujian membuktikan bahwa penambahan jumlah layer recurrent tidak menjamin perolehan skor BLEU-4 dan METEOR yang lebih tinggi.

Variasi	BLEU-4	METEOR	Waktu/gambar
L1_H128	0.046569	0.225452	0.2998s
L2_H128	0.033473	0.198715	0.271495s
L3_H128	0.042764	0.229290	0.217418s

Variasi	BLEU-4	METEOR	Waktu/gambar
L1_H512	0.033290	0.210272	0.325629s
L2_H512	0.039787	0.195868	0.301976s
L3_H512	0.040884	0.221053	0.336624s

2.3.2.2. Perbandingan jumlah layer: LSTM



Gambar 2.3 Training vs Validation Loss LSTM dengan 6 buah variasi

Dari Gambar 2.2, sebenarnya tidak ada perbedaan atau pola yang terlihat dari segi jumlah layer. Baik L1 maupun L2 sama-sama menunjukkan pola konvergensi yang cukup baik. Validation loss juga cenderung stabil setidaknya mulai dari epoch ke-4.

Variasi	BLEU-4	Waktu/gambar
L1_H128	0.0364	0.271s
L2_H128	0.0494	0.364s
L3_H128	0.0343	0.329s

Variasi	BLEU-4	Waktu/gambar
L1_H512	0.0455	0.442s
L2_H512	0.0437	0.559s
L3_H512	0.0540	0.635s

Untuk hidden state 128, model dengan 2 layer (L2_H128) menghasilkan BLEU-4 tertinggi sebesar 0.0494, mengungguli model 1 layer dan 3 layer. Namun untuk hidden state 512, model dengan 3 layer (L3_H512) menghasilkan BLEU-4 tertinggi sebesar 0.0540. Ini menunjukkan bahwa penambahan layer tidak selalu meningkatkan performa secara linear karena terdapat titik optimal yang berbeda tergantung ukuran hidden state. Sementara itu, dari sisi waktu inferensi, penambahan layer meningkatkan waktu eksekusi secara konsisten. L3_H128 membutuhkan waktu 0.329s/gambar dibandingkan L1_H128 yang hanya 0.271s/gambar, sedangkan L3_H512 membutuhkan 0.635s/gambar dibandingkan L1_H512 yang 0.442s/gambar.

Sedikit kesimpulan yang dapat diambil adalah penambahan jumlah layer tidak selalu meningkatkan BLEU-4 score dan selalu meningkatkan waktu inferensi tetapi arsitektur yang optimal bergantung pada ukuran hidden state yang digunakan.

2.3.2.3. Perbandingan ukuran hidden state: RNN

Berdasarkan gambar 2.1, peningkatan ukuran hidden state dari 128 menjadi 512 secara konsisten mempercepat dan memperparah terjadinya

gejala overfitting. Pada seluruh variasi dengan kapasitas H512 (rnn_L1_H512, rnn_L2_H512, dan rnn_L3_H512), kurva validation loss mencapai titik minimum lokal yang optimal di sekitar epoch 5 hingga 6, sebelum akhirnya berbalik naik secara signifikan sementara training loss terus menurun landai. Hal ini menunjukkan bahwa peningkatan dimensi hidden state secara drastis meningkatkan kapasitas parameter model secara berlebihan, membuat model cenderung menghafal deretan teks pada data training alih-alih mempelajari generalisasi fitur spasial gambar.

2.3.2.4. Perbandingan ukuran hidden state: LSTM

Variasi	Hidden 128	Hidden 512	Selisih BLEU-4
1 Layer	0.0364	0.0455	+0.0091
2 Layer	0.0494	0.0437	-0.0057
3 Layer	0.0343	0.0540	+0.0197

Secara umum, hidden state yang lebih besar (512) cenderung menghasilkan BLEU-4 yang lebih tinggi karena terlihat jelas pada variasi 1 layer dan 3 layer. Hal ini masuk akal karena hidden state yang lebih besar memiliki kapasitas representasi yang lebih tinggi untuk menyimpan informasi konteks caption. Akan tetapi, pada variasi 2 layer, hidden state 128 justru sedikit mengungguli hidden state 512 (0.0494 vs 0.0437). Ini kemungkinan disebabkan oleh trade-off antara kapasitas model dan kemampuan generalisasi. Hipotesisnya adalah model yang terlalu besar pada dataset yang relatif kecil (6000 gambar training) dapat mengalami kesulitan generalisasi.

Dari sisi waktu inferensi, hidden state 512 selalu lebih lambat dibandingkan hidden state 128 pada jumlah layer yang sama, karena operasi matrix multiplication yang lebih besar. Dari grafik training/val loss, variasi H512 menunjukkan early stopping yang lebih cepat (7-8 epoch) dibandingkan H128 (9-10 epoch), mengindikasikan model H512 konvergen lebih cepat namun juga lebih rentan overfitting pada epoch lanjut.

Pada percobaan ini, hidden state yang lebih besar umumnya menghasilkan BLEU-4 yang lebih tinggi dengan trade-off waktu inferensi yang lebih lambat. Kombinasi terbaik untuk LSTM adalah L3_H512

dengan BLEU-4 0.0540, meskipun L2_H128 menawarkan keseimbangan yang baik antara performa dan efisiensi.

2.3.2.5. Perbandingan RNN vs LSTM

a. BLEU-4, METEOR score dan Waktu Eksekusi

Aspek	RNN terbaik (L1_H128)	LSTM terbaik (L3_H512)
BLEU-4	0.0466	0.0540
METEOR	0.2255	-
Waktu eksekusi	0.2998s	0.635s

b. Analisis vanishing gradient

Fitur gambar hanya dimasukkan di awal ($t = -1$). Setiap kali model memproses kata baru, memori (h_t) diperbarui menggunakan fungsi tanh. Saat proses training, turunan dari fungsi tanh yang bernilai kecil (<1) dikalikan secara berulang kali sepanjang kalimat. Hal ini membuat nilai gradien mengecil secara drastis hingga habis (mendekati nol) sebelum sempat mundur sampai ke langkah awal. Karena gradiennya hilang, bobot RNN ($W_{\{hh\}}$) gagal mempelajari hubungan jangka panjang. Akibatnya, baru beberapa langkah kata, SimpleRNN sudah lupa total dengan isi gambar. Model kehilangan arah dan tidak lagi mendeskripsikan gambar, melainkan hanya menebak-nebak kata berikutnya secara asal berdasarkan satu atau dua kata terakhir yang baru saja dibuat.

LSTM mengatasi masalah ini dengan menambahkan cell state C_t sebagai jalur memori jangka panjang yang terpisah dari hidden state. Cell state mengalir melalui waktu dengan operasi yang relatif sederhana. Operasi penjumlahan (bukan perkalian berulang) pada cell state menciptakan jalur gradient yang lebih stabil sehingga gradient dapat mengalir mundur tanpa mengecil secara eksponensial. Ini dikenal sebagai constant error carousel.

Dari hasil eksperimen, LSTM terbaik (L3_H512, BLEU-4: 0.0540) mengungguli RNN terbaik (L1_H128, BLEU-4: 0.0466) sebesar +0.0074 poin. Meskipun selisihnya tidak besar, perbedaan ini konsisten dan dapat dijelaskan melalui kemampuan memori jangka

panjang LSTM. LSTM mampu mempertahankan informasi fitur CNN lebih lama sepanjang sequence generation. Ini terlihat dari caption LSTM yang cenderung lebih spesifik terhadap konten gambar dibandingkan RNN yang sering menghasilkan caption generik seperti "a man and a woman are sitting on a bench" untuk berbagai gambar yang berbeda. meskipun LSTM lebih lambat (0.423s/gambar vs 0.202s/gambar untuk implementasi scratch) karena komputasi 4 gate dibandingkan 1 persamaan pada RNN, trade-off ini sebanding dengan peningkatan kualitas caption yang dihasilkan.

c. Qualitative Analysis

Score BLEU: 0.4111

Prediksi RNN:
a man and a woman are sitting on a benchGT: A man and a woman in white shirts and dark glasses smiling
GT: A man and woman wearing sunglasses and white t-shirts smile for the camera
GT: A woman and a man are standing in front of bushes wearing sunglasses

Score BLEU: 0.2069

Prediksi RNN:
a man in a white shirt and a white shirt is standing on a sidewalkGT: A man in a wetsuit is surfing on the ocean
GT: A surfer in a yellow and black wet-suit is falling off a breaking wave.
GT: A surfer on his white surfboard catching a wave.

Score BLEU: 0.0000

Prediksi RNN:
a man and a woman are sitting on a benchGT: A bunch of bikes racing on a track
GT: A group of men on motorcycles round the corner on a racetrack.
GT: A group of motorcycles in a race.

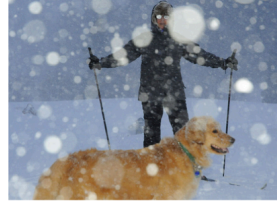
Score BLEU: 0.0000

Prediksi RNN:
a man and a woman are sitting on a benchGT: A child running on a beach with seagulls flying around
GT: "A child runs along the shore," while birds fly overhead -
GT: A small child is running on the shore of the beach surrounded by birds .

Score BLEU: 0.0000

Prediksi RNN:
a man and a woman are sitting on a benchGT: Two dolphins flying headfirst into a beautiful tropical blue lake
GT: Two dolphins jumped out of the water in this zoo
GT: Two dolphins jumping into the water.

Score BLEU: 0.3021

Prediksi RNN:
a man and a woman are sitting on a benchGT: A golden retriever standing outside in the snow with a person standing with skis and poles .
GT: A man and a dog outside in a snow storm
GT: A person is standing on skis in the snow with a dog.

Score BLEU: 0.0000

Prediksi RNN:
a man and a woman are sitting on a benchGT: Five children ride an amusement park ride .
GT: Five girls on a carnival ride.
GT: Five girls on an amusement ride.

Score BLEU: 0.0000

Prediksi RNN:
a man and a woman are sitting on a benchGT: people are walking in an industrial area
GT: An air 's-eye-view of people walking along a street
GT: Many people are walking in the city near tall buildings

Score BLEU: 0.0000

Prediksi RNN:
a man and a woman are sitting on a benchGT: A brown dog is standing in the water .
GT: A brown dog standing in water.
GT: A dog stands in water.

Score BLEU: 0.0000

Prediksi RNN:
a man and a woman are standing on a sidewalkGT: The dogs are running in the water
GT: Three dogs are running through the water
GT: Three dogs running in the water.**Gambar 2.4** Sepuluh foto RNN dengan skor bervariasi

Qualitative Analysis LSTM



Gambar 2.5 Sepuluh foto LSTM dengan skor bervariasi

Model RNN secara konstan membangkitkan kalimat yang hampir sama untuk objek yang sepenuhnya berbeda. Kalimat "a man and a woman are sitting on a bench" muncul sebagai prediksi untuk gambar anjing di atas salju, anak-anak di wahana bermain, balapan motor, kerumunan orang berjalan, anak kecil di pantai, hingga lomba-lomba di kolam. Model mengalami kegagalan total dalam membedakan fitur dan cenderung

mengeluarkan sekuens dominan (safe prediction) yang paling sering muncul pada dataset latih.

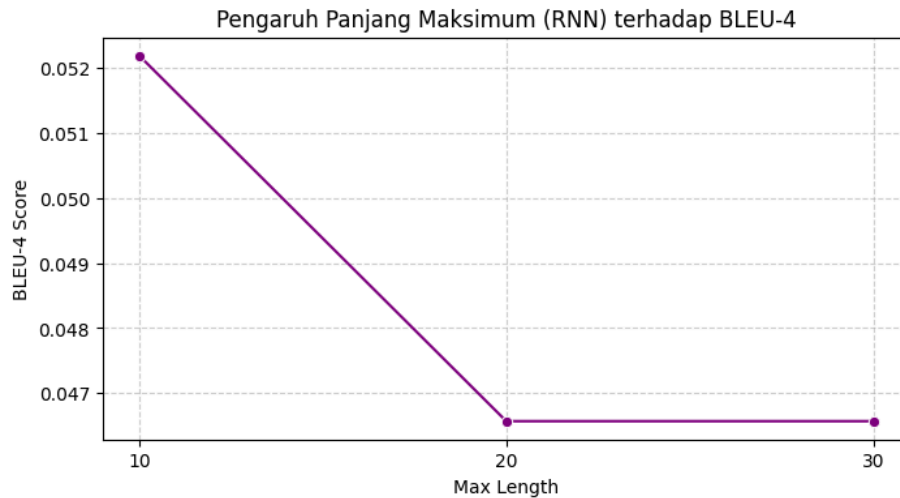
Sementara itu, LSTM menghasilkan caption yang lebih bervariasi meskipun masih sering tidak akurat. LSTM juga sedikit lebih sensitif terhadap fitur visual yang ditunjukkan dengan caption LSTM yang kadang menyebutkan warna baju, jumlah orang, atau objek yang relevan. Lalu, LSTM mencapai BLEU-4 maksimum 0.5206 sedangkan RNN hanya 0.4111 sehingga bisa dikatakan LSTM unggul pada gambar yang kontennya lebih umum dan sesuai dengan distribusi training data.

2.3.2.6. Perbandingan Keras vs from scratch

Arsitektur	RNN terbaik (L1_H128)	LSTM terbaik (L3_H512)
Scratch (Waktu/img)	0.202s	0.423s
Keras (Waktu/img)	1.539s	1.865s

LSTM secara konsisten lebih lambat dibandingkan RNN pada kedua implementasi. Hal tersebut ditunjukkan oleh angka implementasi scratch LSTM membutuhkan waktu 0.423s/gambar, sekitar 2.1x lebih lambat dari RNN scratch (0.202s/gambar). Ini disebabkan oleh komputasi 4 gate (input, forget, output, candidate) pada LSTM dibandingkan hanya 1 persamaan pada SimpleRNN, sehingga jumlah operasi matrix multiplication per timestep LSTM adalah 4x lebih banyak. Namun, hal menarik adalah implementasi scratch lebih cepat dari Keras pada kedua arsitektur. Dari percobaan, RNN scratch 7.6x lebih cepat dari Keras, sedangkan LSTM scratch 4.4x lebih cepat. Ini terjadi karena implementasi Keras memanggil `decoder.predict()` sebanyak `max_len` kali per gambar (greedy decoding yang tidak efisien), sedangkan implementasi scratch langsung loop NumPy tanpa overhead framework.

2.3.2.7. Pengaruh panjang maksimum caption terhadap BLEU-4



Gambar 2.6 Pengaruh panjang maksimum caption terhadap BLEU-4 untuk RNN

Saat max_length kecil (10) Skor BLEU-4 mencapai titik tertinggi karena pembatasan kalimat yang pendek mencegah SimpleRNN mengalami looping kata atau menebak caption dengan asal akibat efek vanishing gradient. Saat max_length diperpanjang (20 dan 30) Skor BLEU-4 mengalami penurunan dan nilainya stagnan, membuktikan bahwa pada timestep ke-20, model RNN sudah selesai membangkitkan kalimat amannya atau sudah terjebak dalam perulangan kata yang tak berujung. Alhasil, menaikkan batas hingga 30 tidak mengubah apa pun karena kualitas kalimatnya sudah rusak sejak timestep sebelumnya.

Max Length	BLEU-4
10	0.0392
20	0.0460
30	0.0455

Sementara itu, LSTM menunjukkan pola peningkatan BLEU-4 seiring bertambahnya max length dari 10 ke 20 (0.0392 menjadi 0.0460), namun tidak ada peningkatan signifikan dari 20 ke 30 (0.0460 menjadi 0.0455). Ini mengindikasikan bahwa sebagian besar caption yang dihasilkan LSTM sudah berhenti sebelum mencapai 20 token karena model sudah menghasilkan token <end>. Menambah max length di atas 20 tidak memberikan manfaat tambahan.

Dari eksperimen ini, didapat hipotesis atau kesimpulan bahwa panjang maksimum yang optimal untuk LSTM adalah 20 karena terbukti memberikan BLEU-4 tertinggi dengan waktu inferensi yang reasonable. Untuk RNN, panjang maksimum yang lebih pendek (10) justru menghasilkan BLEU-4 lebih tinggi, yang secara tidak langsung mencerminkan kelemahan RNN dalam mempertahankan konteks untuk sequence panjang. Ini menjadi salah satu bukti empiris tambahan yang mendukung keunggulan LSTM atas RNN dalam task image captioning.

3. Kesimpulan dan Saran

3.1. Kesimpulan

Implementasi forward propagation scratch mengalami penurunan performa sebesar 0.1741 yang bersumber dari implementasi forward propagation atau layer yang kurang bagus, terutama untuk pemisahan kelas seperti glacier dan buildings.

Konfigurasi Keras terbaik dicapai oleh model CNN_L3_F32_K3_Pmax (Layer = 3, Filter = 32, Kernel = 3×3, Pooling = max) dengan Macro F1 0.8219. Ini karena optimasi 3 layer dan max pooling mempercepat konvergensi, sedangkan penggunaan filter 64 dan kernel 5×5 justru menurunkan performa akibat kejenuhan fitur spasial.

Pengujian shared vs non-shared menghasilkan nilai performa dan distribusi confusion matrix yang identik (0.6478), namun ketiadaan mekanisme weight sharing pada arsitektur non-shared memicu lonjakan parameter drastis hingga 63.95 kali lipat (86.168.518 parameter), menegaskan bahwa parameter sharing pada layer konvolusi standar sangat penting untuk efisiensi memori komputasi tanpa mengurangi kualitas model.

Sementara itu, untuk LSTM dan RNN, didapat fakta bahwa penambahan jumlah layer recurrent tidak selalu meningkatkan performa secara linear. Hidden state yang lebih besar (512) umumnya menghasilkan BLEU-4 yang lebih tinggi dibandingkan hidden state kecil (128) pada jumlah layer yang sama, dengan trade-off waktu inferensi yang lebih lambat. Namun pada beberapa konfigurasi, hidden state kecil justru lebih baik, menunjukkan bahwa kapasitas model yang terlalu besar untuk dataset yang relatif kecil dapat menurunkan kemampuan generalisasi.

Lalu, Implementasi forward propagation from scratch menghasilkan output yang identik dengan implementasi Keras, membuktikan kebenaran implementasi. Menariknya, implementasi scratch lebih cepat dibandingkan Keras (LSTM scratch 4.4x lebih cepat, RNN scratch 7.6x lebih cepat) karena implementasi Keras memanggil predict() sebanyak max_len kali per gambar, sedangkan scratch langsung menggunakan loop NumPy tanpa overhead framework.

Selain itu, LSTM menghasilkan caption yang lebih baik dibandingkan RNN, dengan BLEU-4 terbaik 0.054 (LSTM L3_H512) vs 0.047 (RNN L1_H128). Perbedaan ini disebabkan oleh mekanisme gating LSTM yang memungkinkan model mempertahankan konteks visual dari fitur CNN lebih lama sepanjang sequence generation. Terakhir, Max length optimal untuk LSTM adalah 20 —

memberikan BLEU-4 tertinggi (0.0460) tanpa overhead komputasi yang tidak perlu. Untuk RNN, max length yang lebih pendek (10) justru menghasilkan BLEU-4 lebih tinggi (0.0521), yang secara empiris mendukung kelemahan RNN dalam mempertahankan konteks untuk sequence panjang.

3.2. Saran

Saran yang dapat diberikan untuk pengembangan selanjutnya adalah:

- Attention mechanism dapat digunakan karena arsitektur encoder-decoder sederhana tanpa attention memiliki keterbatasan fundamental
- Fine-tuning CNN encoder karena CNN encoder (InceptionV3) di-freeze sepenuhnya selama training. Fine-tuning beberapa layer terakhir encoder secara bersamaan dengan decoder dapat menghasilkan fitur yang lebih relevan untuk task captioning.
- Optimasi greedy decoding karena implementasi Keras saat ini menggunakan full sequence prediction di setiap timestep yang tidak efisien. Implementasi beam search atau autoregressive decoding yang lebih efisien dapat meningkatkan kualitas caption sekaligus mengurangi waktu inferensi.

4. Pembagian Tugas

NIM	Bagian Yang Dikerjakan
13523005	RNN
13523035	CNN
13523041	LSTM

5. Referensi

- [1]. [d2l.ai: LSTM](#)
- [2]. [d2l.ai: Deep RNN](#)
- [3]. [d2l.ai: Bidirectional RNN](#)
- [4]. [d2l.ai: RNN chapter](#)
- [5]. [d2l.ai: CNN chapter](#)
- [6]. [d2l.ai: Image Captioning](#)
- [7]. [d2l.ai: Beam Search](#)
- [8]. [numpy.einsum](#)
- [9]. [Vinyals et al. \(2015\): Show and Tell: A Neural Image Caption Generator](#)
- [10]. [Tanti et al. \(2017\): Where to put the Image in an Image Caption Generator](#)
- [11]. [Selvaraju et al. \(2016\): Grad-CAM](#)
- [12]. [Flickr8k dataset \(Kaggle\)](#)
- [13]. [Intel Image Classification dataset \(Kaggle\)](#)
- [14]. [Karpathy: The Unreasonable Effectiveness of RNNs](#)
- [15]. [CS231n Lecture 10: RNNs & Image Captioning \(Stanford\)](#)

6. Lampiran

SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:

Nama/NIM : Muhammad Alfansya/13523005
M. Rayhan Farrukh/13523035
Hanif Kalyana Aditya/13523041
Fakultas : Sekolah Teknik Elektro dan Informatika
Kode Mata Kuliah : IF3270
Nama Mata Kuliah : Pembelajaran Mesin
Judul Tugas : Tugas Besar 2 IF3270 Pembelajaran Mesin

Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Adapun bentuk penggunaan dan alat yang digunakan adalah sebagai berikut:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan (Gemini)	Iya	Memeriksa tata bahasa, hasil pemeriksaan diperiksa kembali
Pembuatan Teks	Tidak	
Bantuan Diskusi dalam Penyusunan Konten (Gemini)	Iya	Bantuan memahami konsep materi, memvalidasi kebenaran pemahaman
Lainnya (Gemini, Claude)	Ya	Melakukan debugging, menemukan kode penyebab error. Setelah ditemukan, kode diperbaiki manual

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 16 Mei 2026



Muhammad Alfansya
13523005

Bandung, 16 Mei 2026



M. Rayhan Farrukh
13523035

Bandung, 16 Mei 2026



Hanif Kalyana Aditya
13523041